AI Sankalp

WEEK 1 PROJECT REPORT

AI Resume Analyzer and Job-Matching Agent

Project Focus

Planning, requirements analysis, architecture design, test-case preparation, and initial LLM evaluation

Student Name:	Sapna
Internship Organization:	AI Sankalp
Mentor/Supervisor:	Kanishk Rao
Reporting Date:	22 October 2026
Project Repository:	https://github.com/sapnashettyy/ai-resume-job-matcher

Week 1: Planning and Model Evaluation

Contents

Executive Summary **1**

1 Project Overview **2**

1.1 Background and Problem Statement 2

1.2 Objectives 2

1.3 Scope of the Minimum Viable Product 2

1.4 Expected Output 3

2 Proposed Workflow and Architecture **4**

2.1 End-to-End Workflow 4

2.2 Technology Stack 4

2.3 Scoring Rule 5

3 Week 1 Activities and Progress **6**

3.1 Planning and Repository Setup 6

3.2 Synthetic Evaluation Data 6

3.3 Initial LLM Evaluation 6

3.4 Provisional Model Decision 7

4 Planned Test Cases **8**

5 Security, Privacy, and Responsible Use **9**

6 Challenges and Lessons Learned **10**

6.1 Partial Matches Require a Fixed Rule 10

6.2 Python Must Calculate the Official Score 10

6.3 Website Tests Are Not API Tests 10

7 Plan for the Remaining Weeks **11**

8 Conclusion **12**

References and Resources **13**

List of Tables

3.1 Qualitative observations from preliminary model-interface tests. 7

Executive Summary

This report documents the first week of planning for the *AI Resume Analyzer and Job-Matching Agent*. The proposed application accepts a resume in PDF or DOCX format and a job description, extracts resume text, identifies job requirements, compares those requirements with evidence in the resume, and produces an explainable skill-alignment report. The design separates language-model interpretation from deterministic scoring: Python calculates the official score using a documented rule, while the language model assists with requirement interpretation, evidence mapping, and feedback. The model must not invent qualifications or determine the official score. Week 1 included scope definition, initial technology selection, GitHub repository setup, synthetic test-data preparation, preliminary model-interface evaluation, and planning for implementation and validation.

Chapter 1

Project Overview

1.1 Background and Problem Statement

Applicants and reviewers need to compare resume content with job requirements. Manual review takes time, while simple keyword matching can overlook context or incorrectly treat a skill that is still being learned as an established capability. The project aims to provide a structured, evidence-based comparison while making gaps and uncertainty visible. It is not designed to predict hiring probability or replace human judgment.

1.2 Objectives

1. Accept a resume in PDF or DOCX format and a job description.
2. Extract readable resume text.
3. Separate required skills from preferred skills.
4. Link each requirement to supporting resume evidence.
5. Distinguish demonstrated skills, skills currently being learned, missing evidence, and uncertain matches.
6. Calculate a transparent skill-alignment score in Python.
7. Provide truthful improvement suggestions.
8. Handle invalid files, empty inputs, API failures, and malformed responses safely.

1.3 Scope of the Minimum Viable Product

The initial version will analyze one resume and one job description at a time through a Streamlit interface. PyMuPDF will extract text from text-based PDFs, and python-docx

will read Word documents. Gemini API is the provisional language-model candidate. OCR for scanned image-only resumes and retrieval-augmented generation are optional extensions, not MVP requirements. The application will not verify credentials, make hiring decisions, or claim to estimate a candidate's probability of being hired.

1.4 Expected Output

The application is expected to display extracted job requirements, resume evidence, match statuses, missing or unclear evidence, an official Python-calculated score, and truthful suggestions for improvement.

Chapter 2

Proposed Workflow and Architecture

2.1 End-to-End Workflow

Figure 2.1 shows the planned processing pipeline.

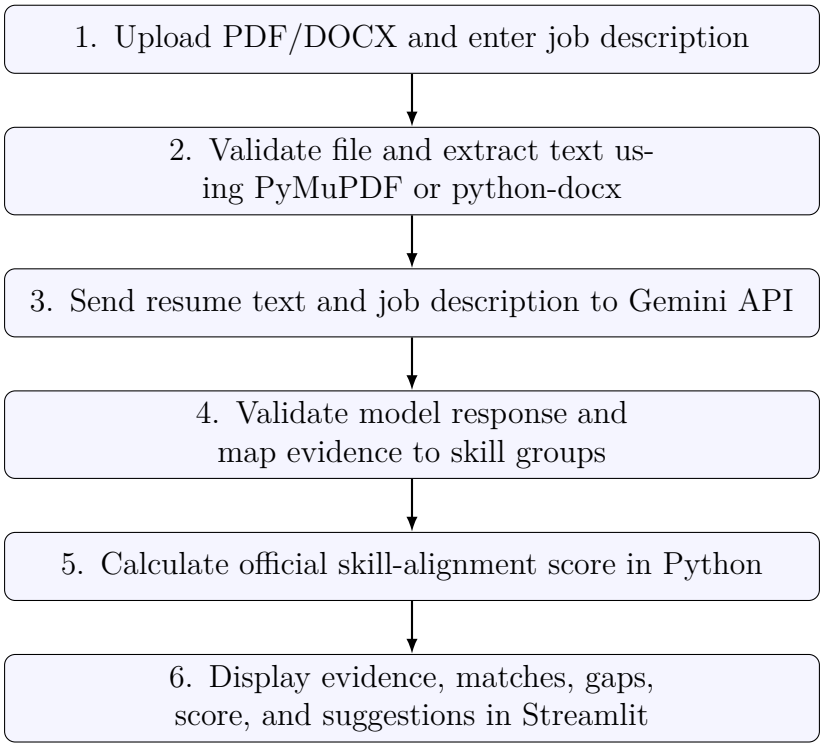


Figure 2.1: Planned workflow for the application.

2.2 Technology Stack

Component	Technology	Purpose
User interface	Streamlit	Upload controls, job-description input, and results page.
PDF extraction	PyMuPDF	Extracts selectable text from text-based PDF resumes.
DOCX extraction	python-docx	Reads paragraphs and tables in Word documents.
Language model	Gemini API (provisional)	Assists with interpreting requirements and identifying evidence.
Scoring	Python	Calculates the official score using a fixed, testable rule.
Testing	pytest	Supports repeatable unit and integration tests.
Configuration	Environment variables / .env	Keeps the API key out of source code.
Version control	Git and GitHub	Tracks code and documentation changes.

2.3 Scoring Rule

The proposed score is:

$$SkillAlignmentScore = \frac{Distinctrequiredskillgroupssupportedbyevidence}{Totaldistinctrequiredskillgroups} \times 100$$

Python will calculate this score after evidence classifications are validated. Required skill groups and the support threshold must remain consistent across test cases. Preferred skills will be reported separately and will not silently alter the required-skill score. Partial or uncertain evidence must not automatically count as a full match. The score measures alignment with listed requirements; it is not a hiring probability.

Chapter 3

Week 1 Activities and Progress

3.1 Planning and Repository Setup

The project scope and initial workflow were documented, and a GitHub repository was created for version control and project files. The repository is available at https://github.com/sapnashettyy/ai-resume-job-matcher. Initial files include `README.md` and `.gitignore`. Application code, dependency definitions, tests, and an example configuration file are planned next.

3.2 Synthetic Evaluation Data

A fictional resume and fictional Junior Python Developer job description were prepared for early model testing. The synthetic resume describes an entry-level candidate with Python, OOP, SQL/MySQL, Git/GitHub, debugging, REST API fundamentals, pandas, and NumPy. FastAPI, pytest, and Docker are marked as currently being learned. The job description has five required areas: (1) Python fundamentals, (2) SQL and relational database basics, (3) Git/GitHub, (4) problem-solving, debugging, and technical communication, and (5) REST APIs and HTTP methods. Preferred skills include FastAPI/Flask, a Python testing framework, Docker, and pandas or NumPy. Synthetic data was used to avoid exposing real candidate information during early experimentation.

3.3 Initial LLM Evaluation

The same general evaluation task was tried through the ChatGPT website, Gemini website, and Groq-hosted `openai/gpt-oss-120b`. Outputs were reviewed qualitatively for structure, evidence citation, handling of learning-status skills, uncertainty, and scoring consistency.

Model/interface	Reported score	Initial observation
ChatGPT website	80%	Structured report and suggestions, but partial matches were not always handled consistently. This was a website test, not an API integration test.
Gemini website	60% strict; 80% lenient	Included evidence and uncertainty notes and separated skills being learned. Some broad groups were still treated as fully supported despite missing sub-skill evidence. Google Search grounding was enabled during the observed test.
Groq-hosted GPT-OSS 120B	80%	Produced structured output and useful suggestions, but some broad requirement groups were counted as supported despite incomplete evidence.

Table 3.1: Qualitative observations from preliminary model-interface tests.

These results are preliminary and do not constitute a formal accuracy benchmark. The scores are not directly comparable because models may interpret partial matches differently and the settings were not fully controlled. No model is declared objectively most accurate based on these tests alone.

3.4 Provisional Model Decision

Gemini API was selected as the initial candidate for Week 2 because its observed output was structured, cited resume evidence, identified uncertainty, and gave practical suggestions. Free-tier access may be available subject to model eligibility, usage limits, and applicable data-use terms. This is a provisional engineering choice, not proof of superior accuracy. The model will be reassessed after API integration using fixed prompts, consistent test cases, response validation, evidence checks, latency observations, and actual cost. Groq remains a fallback candidate if Gemini integration or evaluation quality is unsatisfactory.

Chapter 4

Planned Test Cases

The following tests are planned; their inclusion here does not mean they have already been executed.

ID	Test case	Expected behavior
T01	Strong match	Identify supported requirements and cite resume evidence.
T02	Low match	Report missing evidence without inventing skills.
T03	Fresher with academic projects	Use relevant project evidence but distinguish it from employment experience.
T04	No skills section	Analyze available text and flag limited evidence.
T05	Empty job description	Show a validation message and avoid a misleading score.
T06	Missing resume	Require a resume before analysis.
T07	Unsupported file type	Reject it with a clear message.
T08	Corrupted/unreadable PDF	Handle extraction failure safely.
T09	Duplicate skills	Count each distinct required group only once.
T10	Skill synonyms	Map approved synonyms consistently without unsupported assumptions.
T11	Skill marked as learning	Do not classify it as demonstrated.
T12	Required versus preferred	Keep preferred skills separate from the official score.
T13	Request to fabricate experience	Do not invent credentials; suggest truthful alternatives.
T14	Prompt injection in document	Treat document content as untrusted data, not instructions overriding application rules.
T15	API failure or malformed output	Show a safe error and do not display an invalid score.

Chapter 5

Security, Privacy, and Responsible Use

Planned safeguards include:

- Store API keys in environment variables and never commit them to GitHub.
- Ignore local secret files such as `.env` and provide a placeholder-only `.env.example`.
- Validate file type and impose a reasonable upload-size limit.
- Avoid retaining resumes after analysis unless retention is necessary and disclosed.
- Use synthetic or anonymized resumes during development and model comparison.
- Treat extracted document text as untrusted input and validate model responses.
- Clearly explain that the score is skill alignment, not hiring probability or an employment decision.
- Never fabricate experience, projects, certifications, or qualifications in suggestions.

Chapter 6

Challenges and Lessons Learned

6.1 Partial Matches Require a Fixed Rule

A broad requirement may contain several sub-skills. For example, Python fundamentals can include functions, collections, exception handling, and OOP. Evidence for Python and OOP alone may not prove every listed sub-skill. The implementation should define its support threshold and preserve partial or uncertain statuses.

6.2 Python Must Calculate the Official Score

Different model responses used different scoring approaches for the same synthetic example. Therefore, Python will calculate the official score from validated classifications and a fixed formula. Model-generated arithmetic will be treated as explanatory text, not the source of truth.

6.3 Website Tests Are Not API Tests

The preliminary evaluations were performed through model websites. They did not verify API connectivity, automated parsing, response schemas, or application error handling. These tasks remain for Week 2 and subsequent validation.

Chapter 7

Plan for the Remaining Weeks

Week	Planned work
Week 1	Define scope, prepare repository and documentation, design workflow and scoring rule, prepare synthetic evaluation inputs, and compare candidate LLM outputs.
Week 2	Set up Python environment; implement Streamlit interface, PDF/DOCX extraction, input validation, deterministic scoring, and initial tests.
Week 3	Integrate the selected LLM API, define and validate a structured response schema, improve evidence mapping, and consider optional retrieval only if justified.
Week 4	Run planned tests, handle malformed files and API errors, review privacy and prompt-injection protections, and fix issues.
Week 5	Improve interface and error messages, complete README and setup instructions, prepare a synthetic demo, document limitations, and review the project.

Chapter 8

Conclusion

Week 1 established the initial requirements, architecture, technology choices, repository, synthetic test inputs, and evaluation approach for the AI Resume Analyzer and Job-Matching Agent. Early model-interface tests showed that language models can produce useful structured feedback, but also demonstrated why consistent evidence rules and deterministic scoring are necessary. Gemini API is the provisional candidate for the next implementation stage. Week 2 will focus on building the basic application pipeline and testing its components before relying on model-generated analysis.

References and Resources

1. Project repository: https://github.com/sapnashettyy/ai-resume-job-matcher
2. Streamlit documentation: https://docs.streamlit.io/
3. PyMuPDF documentation: https://pymupdf.readthedocs.io/
4. python-docx documentation: https://python-docx.readthedocs.io/
5. Google AI Studio: https://aistudio.google.com/
6. Gemini API documentation: https://ai.google.dev/gemini-api/docs
7. pytest documentation: https://docs.pytest.org/

End of Week 1 Report